

# Finite-Support Convolutional Likelihoods for Aggregate-Supervised Multiple-Instance Learning

Anonymous Authors

May 6, 2026

## Abstract

Weakly supervised multiple-instance learning often provides supervision only through aggregate labels, such as whether a bag contains a target class or how many target instances it contains. Existing count-based MIL objectives handle Bernoulli counts, but many natural weak labels are richer: signed counts, cancellation-prone aggregates, class histograms, or ordinal sums. We propose a finite-support convolutional view of aggregate likelihoods for Count-MIL. Each instance emits a latent discrete contribution distribution, and the bag-level likelihood is obtained by convolving these atomic probability mass functions. This formulation recovers the standard Bernoulli count likelihood as a special case while extending directly to signed and ordinal aggregate labels. We further derive count-conditioned posterior marginals for instance recovery and implement the likelihood with GPU-parallel convolution, including an FFT-tree backend for large multiclass one-vs-rest count likelihoods. Experiments on MNIST-family bag benchmarks evaluate binary counts, digit-sum supervision, signed cancellation-heavy counts, posterior instance recovery, and multiclass LLP-style histogram supervision, with comparisons to dynamic-programming count loss, attention MIL, expected-sum regression, and proportion-matching baselines. [TODO: Replace final sentence with locked aggregate/CIFAR results.]

## 1 Introduction

Multiple-instance learning (MIL) studies settings where labels are observed at the bag level while instance labels remain latent. In classical binary MIL, a positive bag indicates that at least one instance is positive. Many weak-supervision settings provide more informative aggregate labels: a count of positive instances, a class histogram, a signed count with cancellation, or an ordinal sum. The central problem is to learn an instance-level predictor from these aggregate observations without observing instance labels during training.

This paper studies aggregate-supervised MIL through finite-support likelihoods. Each instance  $x_i$  is mapped to a probability mass function over a finite set of integer contributions. The observed bag label is an aggregate integer value, so the bag likelihood is the coefficient of the corresponding aggregate value in the convolution of the instance-level atomic PMFs. For Bernoulli contributions this recovers the standard count likelihood used in prior count-loss work; the same view directly covers signed counts and ordinal sums.

Our contributions are:

- We formulate Count-MIL as finite-support convolution of instance contribution distributions, recovering Bernoulli count loss as a special case and extending it to signed and ordinal aggregate labels.
- We derive and implement count-conditioned posterior marginals for instance recovery in binary Count-MIL.
- We provide practical GPU implementations, including grouped signed convolution and an FFT-tree backend for large multiclass one-vs-rest count likelihoods.
- We evaluate binary counts, digit sums, signed cancellation-heavy aggregates, FashionMNIST robustness, and multiclass LLP-style histogram supervision.

[TODO: Tighten contribution list after final results are selected.]

## 2 Related Work

**Multiple-instance learning and count loss.** Classical multiple-instance learning (MIL) supervises bags rather than instances [8, 17, 1, 6]. The standard positive-bag assumption is a noisy OR over hidden instance labels: a bag is positive when at least one instance is positive. Neural MIL models learn bag-level predictors end-to-end [24], and attention-based MIL [11] learns a bag representation through learned attention weights. These models are strong baselines for bag prediction, but attention weights are not guaranteed to be calibrated instance posteriors. Count-based extensions use the number of positive instances as a stronger bag label. Shukla et al. [21] formulate count-based weak supervision around the differentiable probability that exactly  $k$  Bernoulli outputs are true, and use dynamic programming to compute the resulting count loss. Related count supervision has also appeared in temporal localization [19]. Recent unified weak-supervision frameworks also cite count loss as a representative count-probability method, but pursue broader latent-label inference problems [25]. Our binary Bernoulli likelihood is not new in this setting: it recovers the count probability used by Shukla et al. We therefore use binary Count-MIL as a sanity check and baseline connection rather than as a novelty claim. Our contribution is the finite-support convolutional generalization and its use for signed, ordinal, posterior, and histogram aggregates.

**Learning from label proportions.** Learning from label proportions (LLP) observes class proportions or histograms over bags [18, 28]. Early and applied work studied probabilistic or application-specific proportion learning [22, 3], while later deep LLP methods considered multiclass training, adversarial training, consistency regularization, pseudo-labeling, and two-stage objectives [10, 9, 13, 23, 14, 27, 15]. Recent theoretical and risk-consistent LLP work includes mutual-contamination reductions, label-noise reductions, unbiased risk estimators, and optimistic-rate analyses [20, 29, 4, 26, 12, 2, 5]. Simple proportion matching trains predicted bag proportions to match observed proportions, but it can over-smooth instance predictions. LLP-PVC [16] reframes label proportions as proportional-value classification and uses efficient divide-and-conquer count likelihood computation. Our histogram one-vs-rest count likelihood is closely related to this PVC view: for each class, the class count is a Bernoulli aggregate over one-vs-rest instance probabilities. This connection is useful but also sets a boundary on our claim. We do not present class-histogram count likelihood as an invention independent of LLP-PVC; instead, we use LLP/PVC-style experiments to position the method and emphasize aggregate labels beyond ordinary class proportions, including scalar ordinal sums and signed cancellation.

**Efficient aggregate inference.** Dynamic programming computes Bernoulli count probabilities exactly but can become expensive for large bags and many classes. Convolutional formulations admit parallel and FFT-based backends [7]. We implement sequential convolution for moderate-size training experiments and an FFT-tree backend for large-bag multiclass one-vs-rest count benchmarks. This makes the computational comparison a backend question: the same aggregate likelihood can be evaluated by dynamic programming, direct convolution, or FFT-tree convolution depending on support size and hardware.

## 3 Method

### 3.1 Finite-Support Aggregate Likelihood

Let a bag contain instances  $x_1, \dots, x_n$ . For each instance, a neural predictor emits a categorical distribution over latent integer contributions:

$$d_i(k) = p_\theta(z_i = k \mid x_i), \quad k \in \{a, \dots, b\}. \quad (1)$$

The observed bag label is an aggregate

$$Y = \sum_{i=1}^n z_i. \quad (2)$$

The exact bag likelihood is the coefficient of  $Y$  in the convolution of atomic PMFs:

$$p_\theta(Y = y \mid x_{1:n}) = (d_1 * d_2 * \dots * d_n)(y). \quad (3)$$

Training minimizes the negative log likelihood

$$\mathcal{L}_{\text{agg}}(\theta) = -\log p_{\theta}(Y = y \mid x_{1:n}). \quad (4)$$

The support of the aggregate PMF is finite and contiguous:

$$\text{supp}(Y) = \{na, na + 1, \dots, nb\}. \quad (5)$$

All experiments use exact likelihoods over this support; approximations enter only through the neural instance model, not through sampling latent assignments.

### 3.2 Training Objective

For each minibatch, the instance model first produces atomic PMFs for every instance in every bag. The aggregate layer then deterministically maps these atoms to an exact PMF over the observed bag label. The training loss is the negative log probability assigned to the observed aggregate,

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{b=1}^B \log(d_{b1} * \dots * d_{bn_b})(y_b). \quad (6)$$

Gradients are backpropagated through the convolutional aggregate layer into the instance network. Instance labels are never used for training in the weak-supervision experiments; they are used only for evaluation of hidden instance recovery. This is different from expected-value regression baselines, which regress the scalar expectation  $\sum_i \mathbb{E}[z_i]$  to the aggregate label and therefore discard the higher-order distributional information in the count or sum.

### 3.3 Special Cases

**Binary counts.** For Bernoulli target indicators,

$$d_i = (1 - p_i, p_i), \quad Y = \sum_i z_i. \quad (7)$$

Equivalently, each instance contributes a length-two one-dimensional kernel over support  $\{0, 1\}$ :

$$\kappa_i = [1 - p_i, p_i]. \quad (8)$$

The probability of observing count  $y$  is the coefficient of  $t^y$  in

$$\prod_{i=1}^n ((1 - p_i) + p_i t). \quad (9)$$

This recovers the standard binary count likelihood.

**Signed counts.** For known signs  $s_i \in \{-1, +1\}$  and latent target indicators  $z_i \in \{0, 1\}$ ,

$$Y = \sum_i s_i z_i. \quad (10)$$

Each atom has support  $\{-1, 0, +1\}$  after incorporating the sign:

$$d_i = \begin{cases} (1 - p_i)\delta_0 + p_i\delta_{+1}, & s_i = +1, \\ (1 - p_i)\delta_0 + p_i\delta_{-1}, & s_i = -1. \end{cases} \quad (11)$$

In array order  $[-1, 0, +1]$ , the corresponding signed kernels are

$$\kappa_i^{(+)} = [0, 1 - p_i, p_i], \quad \kappa_i^{(-)} = [p_i, 1 - p_i, 0]. \quad (12)$$

This setting exposes cancellation: many distinct latent assignments can yield the same observed aggregate, especially when  $Y = 0$ .

**Ordinal sums.** For hidden ordinal values  $z_i \in \{0, \dots, K-1\}$ , the observed label can be a scalar sum such as a digit sum. Unlike expected-sum regression, the likelihood uses the full induced distribution over possible sums. If a model predicts digit probabilities  $p_\theta(z_i = k \mid x_i)$ , the likelihood compares the observed sum against the full convolved PMF rather than only the mean  $\sum_i \mathbb{E}[z_i]$ . The one-dimensional kernel for instance  $i$  is the full class-probability vector

$$\kappa_i = [p_{i0}, p_{i1}, \dots, p_{i,K-1}], \quad (13)$$

so the aggregate support has width  $n(K-1) + 1$ . For MNIST digit-sum,  $K = 10$  and the observed bag label is the sum of hidden digit values.

**Histogram labels and one-vs-rest counts.** For multiclass histograms, each class count can be treated as a one-vs-rest Bernoulli count. Let  $p_{ic}$  be the predicted probability of class  $c$  for instance  $i$ , and let  $h_c$  be the observed count of class  $c$  in the bag. For every class  $c$ , we form Bernoulli count kernels

$$\kappa_{ic} = [1 - p_{ic}, p_{ic}] \quad (14)$$

and compute a classwise count likelihood. The one-vs-rest count objective is

$$\mathcal{L}_{\text{hist}}(\theta) = -\frac{1}{C} \sum_{c=1}^C \log \left[ \prod_{i=1}^n ((1 - p_{ic}) + p_{ic} t) \right]_{t^{h_c}}, \quad (15)$$

where  $[\cdot]_{t^{h_c}}$  denotes coefficient extraction. This yields a PVC-style objective closely related to LLP-PVC, while sharing the same finite-support convolution backend.

### 3.4 Posterior Marginals

For binary counts, we compute count-conditioned posterior marginals

$$q_i = p_\theta(z_i = 1 \mid \sum_j z_j = y, x_{1:n}), \quad (16)$$

using prefix/suffix count distributions. Let  $F_{i-1}(k)$  be the probability that the first  $i-1$  instances contain  $k$  positives, and let  $B_{i+1}(k)$  be the probability that instances  $i+1, \dots, n$  contain  $k$  positives. Then

$$q_i = \frac{p_i \sum_k F_{i-1}(k) B_{i+1}(y-1-k)}{\sum_k F_{i-1}(k) ((1-p_i) B_{i+1}(y-k) + p_i B_{i+1}(y-1-k))}. \quad (17)$$

These marginals are used as additional information extracted from the count label after likelihood training. They support instance recovery diagnostics, uncertainty analysis, and optional pseudo-label inspection, but they are not part of the core training objective in this paper.

### 3.5 Computational Backends

All of the above objectives are one-dimensional convolutions over integer aggregate support. If  $m^{(i)}$  denotes the aggregate PMF after processing the first  $i$  atoms, then

$$m^{(i)}(s) = \sum_{r=a}^b m^{(i-1)}(s-r) d_i(r), \quad m^{(0)} = \delta_0. \quad (18)$$

This recurrence is the same probability law whether implemented as dynamic programming, shifted dense tensor accumulation, grouped `conv1d`, or FFT polynomial multiplication. The implementation uses several exact backends:

- Sequential finite-support convolution accumulates shifted copies of the current PMF and is used for moderate binary, signed, and ordinal-sum experiments.

Table 1: Exact aggregate backends.  $n$  is bag size,  $K$  is atomic support width,  $S = n(K - 1) + 1$  is aggregate support width, and  $C$  is number of one-vs-rest classes.

| Backend                | Setting              | Work per bag             | Use in this paper                       |
|------------------------|----------------------|--------------------------|-----------------------------------------|
| DP                     | Bernoulli count      | $O(nS)$                  | Shukla-style count baseline             |
| Sequential convolution | finite support       | $O(nKS)$                 | main moderate-bag experiments           |
| Grouped convolution    | signed Bernoulli     | $O(nS)$ , batched groups | signed GPU implementation               |
| FFT-tree               | finite support / OVR | $O(CS \log S \log n)$    | large-bag runtime, PVC-style histograms |

- Dynamic programming is used as a Shukla-style Bernoulli count baseline; it computes the same binary count coefficients as the convolutional view.
- Grouped GPU convolution packs signed Bernoulli groups into channels and applies grouped `conv1d`.
- Balanced FFT-tree convolution multiplies the atomic polynomials pairwise and is used for large-bag runtime experiments and multiclass one-vs-rest histogram/PVC likelihoods.

**Grouped signed `conv1d`.** The literal GPU `conv1d` backend is used for grouped signed binary aggregates. Each active group starts as a point mass at zero on support  $[-K_{\max}, K_{\max}]$ . For one signed instance, the backend applies a length-three channelwise kernel and padding one. Because PyTorch’s `conv1d` operator is cross-correlation rather than mathematical convolution, the code stores the kernel in the order

$$[p_i \mathbf{1}\{s_i = +1\}, 1 - p_i, p_i \mathbf{1}\{s_i = -1\}], \quad (19)$$

so the left kernel entry shifts probability mass toward larger aggregate values and the right entry shifts mass toward smaller values. This is only an implementation orientation; in mathematical support order  $[-1, 0, +1]$  the signed kernels are the atoms shown above.

**Multiclass and histogram implementation.** For digit-sum, the atom width is the number of ordinal labels, e.g. ten for MNIST digits. For histogram/PVC experiments, we avoid a full  $C$ -dimensional joint histogram over classes and instead compute  $C$  one-vs-rest one-dimensional count likelihoods in parallel. This keeps the aggregate support one-dimensional for each class and makes large bags practical on GPU. The FFT-tree backend is especially useful for this setting because the same pairwise polynomial products are batched over classes and bags. Padding or masked positions are represented by deterministic zero-contribution atoms, such as  $[1, 0]$  for Bernoulli counts,  $[0, 1, 0]$  for signed support  $[-1, 0, +1]$ , and  $[1, 0, \dots, 0]$  for ordinal sums.

## 4 Experiments

### 4.1 Experimental Setup

We evaluate aggregate-supervised instance learning under five groups of experiments:

1. exactness and runtime microbenchmarks;
2. binary MNIST-Bags count supervision;
3. MNIST digit-sum supervision;
4. signed Count-MIL with cancellation-heavy bags;
5. multiclass histogram LLP/PVC comparisons and CIFAR pilots.

Unless otherwise stated, MNIST-family experiments use LeNet-style instance networks, bags are sampled on the fly, and we report means over three random seeds. Hidden instance labels are never used for training; they are used only for evaluation.

Table 2: Binary MNIST-Bags. Count likelihood recovers strong bag and instance predictors.

| Bag size / train bags | Objective | Bag AUC | Instance AUC | Count MAE |
|-----------------------|-----------|---------|--------------|-----------|
| 10/1000               | NLL       | 1.000   | 0.999        | 0.115     |
| 10/5000               | NLL       | 1.000   | 0.999        | 0.090     |
| 50/1000               | NLL       | 1.000   | 0.998        | 0.686     |
| 50/5000               | NLL       | 1.000   | 0.999        | 0.460     |

Table 3: MNIST digit-sum with exact finite-support likelihood. The model observes only the scalar bag sum during training.

| Bag size / train bags | Instance digit accuracy | Sum MAE |
|-----------------------|-------------------------|---------|
| 10/1000               | 0.982                   | 0.612   |
| 10/5000               | 0.992                   | 0.277   |
| 50/1000               | 0.693                   | 4.901   |
| 50/5000               | 0.887                   | 2.229   |

## 4.2 Binary CountMIL

Plain count-likelihood training recovers high hidden-instance AUC across bag sizes. We tested posterior-based training variants during development, but they did not improve the main likelihood result reliably and are not treated as part of the core method.

## 4.3 Digit-Sum Supervision

Initial results show that exact finite-support likelihood recovers hidden digit predictors far better than expected-value matching. For MNIST bags with  $n = 10$ , exact likelihood achieves about 98–99% hidden digit accuracy. The partial 3090 expected-sum baseline results show that matching only the expected aggregate is much weaker in the low-data regime, but can become competitive for small bags when the number of training bags increases.

The expected-sum baselines running on the 3090 server are designed to isolate whether matching only  $\sum_i \mathbb{E}[z_i]$  is sufficient. Partial completed results are shown in Table 4. For  $n = 10$  and 1000 training bags, expected-value matching yields expected-sum MAE around 2.3–2.5 and hidden digit accuracy around 22–29%, far below the exact likelihood in Table 3. With 5000 training bags, however, MAE and Huber baselines improve sharply and recover high instance accuracy for this small-bag setting. We therefore interpret the result as evidence for the data efficiency of exact scalar-sum likelihood, not as evidence that expected-value baselines universally fail.

## 4.4 Signed CountMIL

Signed random bags are easy for the likelihood; cancellation-heavy bags are substantially harder but still yield strong instance ranking, especially with more training bags.

## 4.5 FashionMNIST Robustness

FashionMNIST confirms that attention baselines can achieve strong bag prediction while providing much weaker hidden-instance recovery than count likelihood training. The A5000 FashionMNIST grid completed without errors. Binary CountMIL achieves hidden-instance AUC around 0.997–0.998, while attention and gated-attention baselines reach strong bag prediction but only about 0.70–0.74 instance AUC. Signed CountMIL also preserves high instance AUC around 0.996–0.998. Digit-sum is weaker on FashionMNIST, whereas histogram LLP is much stronger for category recovery. We therefore use FashionMNIST primarily as a robustness result for instance recovery rather than as the cleanest ordinal-sum demonstration.

Table 4: Partial expected-sum regression baselines on MNIST digit-sum bags from the 3090 server. The  $n = 10$ , train 5000 rows currently contain one completed seed and should be replaced by final averages when available.

| Bag/train | Loss  | Seeds | Exp. MAE | Rounded acc. | Digit acc. |
|-----------|-------|-------|----------|--------------|------------|
| 10/1000   | Huber | 3     | 2.330    | 0.136        | 0.278      |
| 10/1000   | MAE   | 3     | 2.337    | 0.141        | 0.286      |
| 10/1000   | MSE   | 3     | 2.535    | 0.133        | 0.226      |
| 10/5000   | Huber | 1     | 0.548    | 0.784        | 0.988      |
| 10/5000   | MAE   | 1     | 0.430    | 0.835        | 0.989      |
| 10/5000   | MSE   | 1     | 1.022    | 0.386        | 0.730      |

Table 5: Signed MNIST Count-MIL. Cancellation-heavy bags are substantially harder, especially for large bags with limited training data.

| Bag size / train bags | Signs        | Instance AUC | Signed MAE | Zero-count accuracy |
|-----------------------|--------------|--------------|------------|---------------------|
| 10/1000               | random       | 0.995        | 0.095      | 0.933               |
| 10/1000               | cancellation | 0.998        | 0.071      | 0.965               |
| 10/5000               | random       | 0.999        | 0.045      | 0.969               |
| 10/5000               | cancellation | 0.999        | 0.046      | 0.979               |
| 50/1000               | random       | 0.999        | 0.243      | 0.772               |
| 50/1000               | cancellation | 0.969        | 1.689      | 0.420               |
| 50/5000               | random       | 0.999        | 0.198      | 0.814               |
| 50/5000               | cancellation | 0.993        | 1.118      | 0.591               |

## 4.6 Histogram LLP and PVC

Histogram supervision is much richer than scalar sums, and LLP baselines are strong on MNIST-family data. We use these experiments to clarify where exact aggregate likelihood adds value and where existing LLP-style objectives are already competitive. The completed 4090 histogram grid is shown in Table 6. KL and CE proportion matching are strong, but the one-vs-rest count likelihood/PVC objective consistently improves both hidden digit recovery and histogram count MAE. The advantage is clearest for larger bags: at  $n = 50$  and 1000 training bags, PVC improves digit accuracy from 96.4% to 98.5% and reduces count MAE from 0.376 to 0.137. MSE proportion matching is not competitive for large bags.

## 4.7 CIFAR Experiments

CIFAR-10 with ResNet-18 is the direct comparison setting for LLP-PVC. Ma et al. report CIFAR-10 results with a ResNet-18 backbone, and their appendix specifies ImageNet-pretrained initialization. We therefore evaluate both randomly initialized and ImageNet-pretrained ResNet-18 backbones because the representation source has a large effect on LLP performance. The from-scratch run completes the same pipeline without external pretrained features, while the pretrained run matches the backbone initialization specified for LLP-PVC.

The completed from-scratch results are much weaker than LLP-PVC, showing that the CIFAR comparison is not meaningful unless the backbone initialization protocol is matched. In contrast, the running pretrained PVC experiment already reaches about 90% instance accuracy for bag sizes 16 and 64 in early epochs, above the reported LLP-PVC accuracies for those bag sizes. We treat these pretrained numbers as promising but provisional until all 500-epoch jobs finish and final checkpoint metrics are aggregated.

CIFAR-100 coarse-histogram pilots currently show nontrivial learning above chance but are not yet strong enough as headline results. The completed CIFAR-100 coarse pilot used 20 coarse classes, bag size 50, train bags 1000, and two seeds. PVC and KL reach about 30% hidden coarse-class accuracy, clearly above the 5% random baseline, but the result is not yet competitive enough for a main claim. We therefore use CIFAR-10 for direct LLP-PVC comparison and treat CIFAR-100 as a harder exploratory setting.

Table 6: Completed MNIST histogram supervision results on the 4090 server. Entries report count MAE / hidden digit accuracy over three seeds. CE and KL give identical results up to rounding in this grid.

| Bag/train | CE/KL     | MSE        | PVC       | PVC hist. exact |
|-----------|-----------|------------|-----------|-----------------|
| 10/1000   | .080/.973 | .181/.955  | .032/.983 | .838            |
| 10/5000   | .030/.989 | .129/.974  | .016/.992 | .918            |
| 50/1000   | .376/.964 | 1.683/.116 | .137/.985 | .436            |
| 50/5000   | .184/.988 | 1.684/.116 | .080/.992 | .659            |

Table 7: CIFAR-10 histogram supervision with ResNet-18. From-scratch results are complete. The ImageNet-pretrained PVC rows follow the backbone initialization specified in the LLP-PVC appendix, but are preliminary snapshots from the running 500-epoch experiment and should be replaced by final checkpoint results.

| Bag size | Backbone/objective      | Hist. MAE      | Prop. MAE       | Instance acc.  |
|----------|-------------------------|----------------|-----------------|----------------|
| 16       | scratch KL              | 0.806          | 0.0522          | 0.513          |
| 16       | scratch MSE             | 0.825          | 0.0533          | 0.476          |
| 16       | scratch PVC             | 0.838          | 0.0524          | 0.527          |
| 64       | scratch KL              | 1.856          | 0.0292          | 0.490          |
| 64       | scratch MSE             | 2.013          | 0.0316          | 0.270          |
| 64       | scratch PVC             | 1.855          | 0.0290          | 0.496          |
| 128      | scratch KL              | 2.860          | 0.0224          | 0.457          |
| 128      | scratch MSE             | 2.829          | 0.0222          | 0.226          |
| 128      | scratch PVC             | 2.707          | 0.0211          | 0.488          |
| 16       | pretrained PVC, running | $\approx 0.26$ | $\approx 0.016$ | $\approx 0.90$ |
| 64       | pretrained PVC, running | $\approx 0.69$ | $\approx 0.011$ | $\approx 0.90$ |
| 16       | LLP-PVC reported        | –              | –               | 0.845          |
| 64       | LLP-PVC reported        | –              | –               | 0.789          |
| 128      | LLP-PVC reported        | –              | –               | 0.760          |

## 4.8 Runtime

Our FFT-tree backend computes batched multiclass one-vs-rest count PMFs on GPU and gives large practical speedups over CPU dynamic programming in large-bag benchmarks.

## 5 Discussion

The experiments support a hierarchy of aggregate supervision. Scalar expected-value matching is weak in low-data digit-sum settings, whereas exact aggregate likelihood can recover accurate instance predictors from the same scalar labels. With more training bags and small bags, expected-value baselines can become competitive, so the strongest claim is data efficiency and likelihood fidelity rather than universal dominance. Histogram labels are substantially richer, and standard LLP/PVC-style baselines can already be strong. On MNIST histogram supervision, however, exact one-vs-rest count likelihood improves over CE/KL proportion matching, especially for larger bags, which suggests that likelihood fidelity still matters even when the aggregate label is rich. This distinction is important for positioning: our goal is not to replace every LLP method, but to provide a unified likelihood framework for aggregate labels beyond ordinary class proportions.

Several limitations remain. Large bags can expose calibration issues even when instance ranking is accurate. Cancellation-heavy signed aggregates are genuinely ambiguous and require more data. CIFAR-scale experiments require stronger backbones and training protocols for fair comparison to recent LLP work. [TODO: Update limitations after final CIFAR-10 ResNet-18 results.]



Table 8: Runtime for multiclass one-vs-rest count PMFs with batch size 256 and  $C = 10$  classes. CPU DP is compared to GPU FFT-tree convolution.

| Bag size | CPU DP (s) | GPU FFT-tree (s) | Max abs. error       |
|----------|------------|------------------|----------------------|
| 64       | 8.246      | 0.00097          | $3.4 \times 10^{-7}$ |
| 128      | 17.230     | 0.00097          | $4.2 \times 10^{-7}$ |
| 256      | 36.785     | 0.00121          | $5.1 \times 10^{-7}$ |
| 512      | 54.040     | 0.00134          | $6.6 \times 10^{-7}$ |

## References

- [1] Stuart Andrews, Ioannis Tsochantaridis, and Thomas Hofmann. Support vector machines for multiple-instance learning. In *Advances in Neural Information Processing Systems*, 2002.
- [2] Lorne Applebaum, Travis Dick, Claudio Gentile, Haim Kaplan, and Tomer Koren. Optimal learning from label proportions with general loss functions, 2025.
- [3] Gerda Bortsova, Florian Dubost, Silas Orting, Ioannis Katramados, Laurens Hogeweg, Laura Thomsen, Mathilde Wille, and Marleen de Bruijne. Deep learning from label proportions for emphysema quantification. *Medical Image Computing and Computer Assisted Intervention*, pages 768–776, 2018.
- [4] Robert Busa-Fekete, Heejin Choi, Travis Dick, Claudio Gentile, and Andres Munoz Medina. Easy learning from label proportions. In *Advances in Neural Information Processing Systems*, pages 14957–14968, 2023.
- [5] Robert Busa-Fekete, Travis Dick, Claudio Gentile, Haim Kaplan, Tomer Koren, and Uri Stemmer. Nearly optimal sample complexity for learning with label proportions, 2025.
- [6] Marc-Andre Carbonneau, Veronika Cheplygina, Eric Granger, and Ghyslain Gagnon. Multiple instance learning: A survey of problem characteristics and applications. *Pattern Recognition*, 77:329–353, 2018.
- [7] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [8] Thomas G. Dietterich, Richard H. Lathrop, and Tomas Lozano-Perez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1–2):31–71, 1997.
- [9] Gabriel Dulac-Arnold, Neil Zeghidour, Marco Cuturi, Lucas Beyer, and Jean-Philippe Vert. Deep multi-class learning from label proportions, 2019.
- [10] Yen-Chang Hsu, Zhaoyang Lv, Joel Schlosser, Phillip Odom, and Zsolt Kira. Multi-class classification without multi-class labels. In *International Conference on Learning Representations*, 2019.
- [11] Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *International Conference on Machine Learning*, 2018.
- [12] Gene Li, Lin Chen, Adel Javanmard, and Vahab Mirrokni. Optimistic rates for learning from label proportions. In *Conference on Learning Theory*, pages 3437–3474, 2024.
- [13] Jiabin Liu, Bo Wang, Zhiquan Qi, Yingjie Tian, and Yong Shi. Learning from label proportions with generative adversarial networks. In *Advances in Neural Information Processing Systems*, 2019.
- [14] Jiabin Liu, Bo Wang, Xin Shen, Zhiquan Qi, and Yingjie Tian. Two-stage training for learning from label proportions. In *International Joint Conference on Artificial Intelligence*, pages 2737–2743, 2021.
- [15] Tianhao Ma, Han Chen, Juncheng Hu, Yungang Zhu, and Ximing Li. Forming auxiliary high-confident instance-level loss to promote learning from label proportions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20592–20601, 2025.

- [16] Tianhao Ma, Wei Wang, Ximing Li, Gang Niu, and Masashi Sugiyama. Learning from label proportions via proportional value classification. In *International Conference on Learning Representations*, 2026.
- [17] Oded Maron and Tomas Lozano-Perez. A framework for multiple-instance learning. In *Advances in Neural Information Processing Systems*, 1998.
- [18] Novi Quadrianto, Alex J. Smola, Tiberio S. Caetano, and Quoc V. Le. Estimating labels from label proportions. *Journal of Machine Learning Research*, 10:2349–2374, 2009.
- [19] Julian Schroeter, Kirill A. Sidorov, and David Marshall. Weakly-supervised temporal localization via occurrence count learning. In *International Conference on Machine Learning*, pages 5649–5659, 2019.
- [20] Clayton Scott and Jianxin Zhang. Learning from label proportions: A mutual contamination framework. In *Advances in Neural Information Processing Systems*, volume 33, pages 22256–22267, 2020.
- [21] Vinay Shukla, Zhe Zeng, Kareem Ahmed, and Guy Van den Broeck. A unified approach to count-based weakly supervised learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 38709–38722, 2023. URL [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/79a0c8e7ae8e403e39341ea6b0ba4c21-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/79a0c8e7ae8e403e39341ea6b0ba4c21-Abstract-Conference.html).
- [22] Tao Sun, Dan Sheldon, and Brendan O’Connor. A probabilistic approach for learning with label proportions applied to the us presidential election. In *IEEE International Conference on Data Mining*, pages 445–454, 2017.
- [23] Kuen-Han Tsai and Hsuan-Tien Lin. Learning from label proportions with consistency regularization. In *Asian Conference on Machine Learning*, pages 513–528, 2020.
- [24] Xinggang Wang, Yongluan Yan, Peng Tang, and Wenyu Liu. Revisiting multiple instance neural networks. *Pattern Recognition*, 74:15–24, 2018.
- [25] Ziquan Wang, Haobo Wang, Ke Chen, Lei Feng, and Gang Chen. FastBUS: A fast bayesian framework for unified weakly-supervised learning, 2026.
- [26] Zixi Wei, Lei Feng, Bo Han, Tongliang Liu, Gang Niu, Xiaofeng Zhu, and Heng Tao Shen. A universal unbiased method for classification from aggregate observations. In *International Conference on Machine Learning*, pages 36804–36820, 2023.
- [27] Haoran Yang, Wanjing Zhang, and Wai Lam. A two-stage training framework with feature-label matching mechanism for learning from label proportions. In *Asian Conference on Machine Learning*, pages 1461–1476, 2021.
- [28] Felix X. Yu, Krzysztof Choromanski, Sanjiv Kumar, Tony Jebara, and Shih-Fu Chang. On learning from label proportions, 2014.
- [29] Jianxin Zhang, Yutong Wang, and Clayton Scott. Learning from label proportions by learning with label noise. In *Advances in Neural Information Processing Systems*, pages 26933–26942, 2022.

## A Additional Derivations

[TODO: Add prefix/suffix posterior derivation for binary count marginals.]

## B Implementation Details

[TODO: List architectures, optimizer settings, bag sampling settings, and hardware.]

## C Additional Tables

[TODO: Add full seed-level tables and exploratory posterior-training ablations if space permits.]